Praktikum 9
Siklus Pelaksanaan (Execution)

NRP	: 1. 3223600038
	2. 3223600041
	3. 3223600048
	4. 3223600050
	5. 3223600058
Nama	: 1. Karolus Wira Wicaksana
	2. Akmal Rafiuddin
	3. Moch. Faris Afif
	4. Labib Al Afaf
	5. Ihsanta Zaki Sanjaya
Kelas	: 2 Teknik Komputer B
Matakuliah	: Praktikum Organisasi Mesin dan Bahasa Assembly
Tanggal	: Rabu, 30 April 2025
Dosen	: Dr. Setiawardhana, ST. MT

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Set Instruction SAP-1

9.1. Tujuan:

a. Mahasiswa dapat menjelaskan dan membuat program Siklus Pelaksanaan

b. Mahasiswa dapat menjelaskan dan membuat program rutin LDA, ADD, SUB, OUT, HLT.

9.2. Dasar Teori:

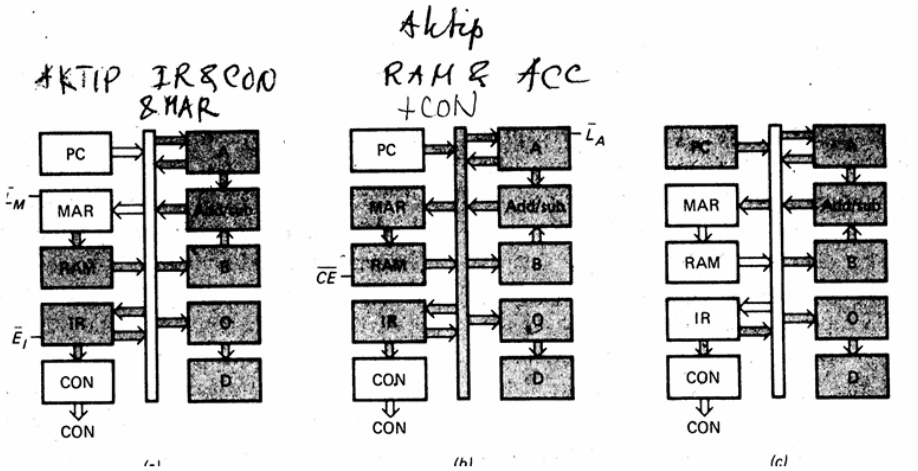
Tiga keadaan berikutnya (T4, T5, dan T6) merupakan siklus eksekusi dari SAP-1. Transfer-transfer register yang terjadi selama siklus eksekusi bergantung pada macam instruksi yang sedang dieksekusi. Misalny, LDA 9H memerlukan operasi transfer register yang berbeda daripada ADD BH. Apa yang diuraikan berikut ini adalah rutin kendali (control routine), artinya rangkaian langkah kendali bagi instruksi-instruksi SAP-1 yang berbeda.

A. Rutin LDA

Demi kejelasan pembahasan, kita mengandaikan bahwa register intruksi (IR) telah diisi dengan instruksi LDA 9H:

IR – 0000 1001

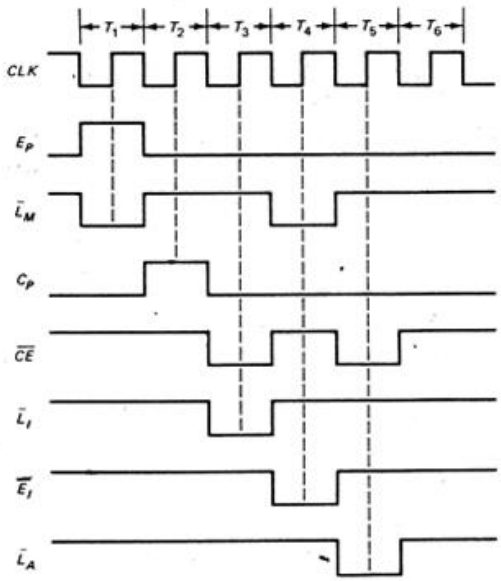
Selama keadan T4, medan instruksi 0000 dikirim ke dalam pengendali-pengurut yang melakukan pendekodean; sedangkan medan alamat 1001 diisikan ke dalam MAR, Gambar 4a memperlihatkan bagian-bagian yang aktif dari SAP-1 selama keadaan T4. Perlu dicatat bahwa E1 dan LM1 adalah bit-bit yang aktif; dan bit-bit kendali yang lain tidak aktif



Gambar 4 Rutin LDA (a) Keadaan T_4 (b) Keadaan T_5 (c) Keadaan T_6

Selama keadaan T5 bit-bit CE dan LA menjadi aktif. Ini berarti kata data yang telah dilamatkan dalam memori akan diisi kedalam akumulator pada tepi positif pulsa detak yang berikutnya (lihat gambar 4b).

T6 adalah keadaan tanpa operasi (no-operation; disingkat Nop). Selama keadaan eksekusi yang ketiga ini semua register dalam kondisi tidak aktif (gambar 4c). Ini berarti bahwa bagian pengendali-pengurut mengeluarkan kata dengan bit-bit yang bersesuaian dengan keadaan tidak aktif. Keadaan T6 dari rutin LDA adalah keadaan Nop.



Gambar 5 Diagram Pewaktuan Siklus Pengambilan LDA

Gambar 5 memperlihatkan diagram pewaktuan untuk rutin pengambilan dan rutin LDA. Selama keadaan T1, Ep dan L Mmenjadi aktif : dan tepi positif detak pada pertengahan keadaan ini akan memindahkan alamat didalam pencacah program kedalam MAR. Selama keadaan T2, CP yang aktif dan hitungan pada pencacah program dinaikkan Pada waktu tibanya tepi positif. Selama keadaan T3, CE dan L 1yang aktif; ketika muncul tepi positif dari sinyal pewaktu, kata RAM yang ditunjuk alamatnya dipindahkan ke register instruksi. Eksekusi LDA dimulai dengan keadaan T4, dimana L Mdan E 1yang aktif . Pada tepi positif detak, medan alamat didalam register instruksi dipindahkan ke MAR. Selama keadaan T5, CE dan L A menjadi aktif; ini berarti kata data RAM dengan alamat yang ditentukan itu ditransfer kepada akumulator pada tepi positif detak. Sebagaimana diketahui keadaan T6 dari rutin LDA adalah keadaan nop.

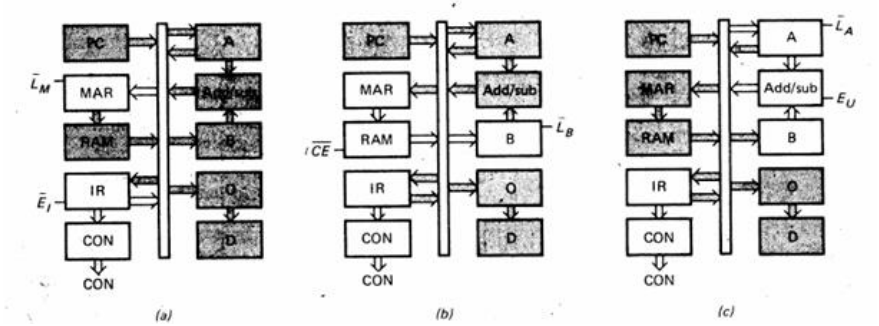
.

B. Rutin ADD

Umpamakan bahwa pada akhir siklus pengambilan register instruksi berisi kata ADD BH.

IR – 0001 1011

Selama keadaan T4 medan instruksi memasuki pengendali-pengurut dan medan alamat berpindah ke MAR (lihat gambar 6a). Dalam keadaan ini E1 dan LM yang aktif.



Gambar 6 Rutin ADD dan SUB (a) Keadaan T_4 (b) Keadaan T_5 (c) Keadaan T_6

Bit-bit kendali dan CE L Bmenjadi aktif selama keadaan T5. Ini memungkinkan kata RAM yang telah ditunjuk alamatnya itu untuk mempersiapkan register B (gambar 6b). Seperti biasa, pengisian terjadi pada pertengahan keadaan ketika tepi positif detak memasuki saluran-saluran CLK dari register B.

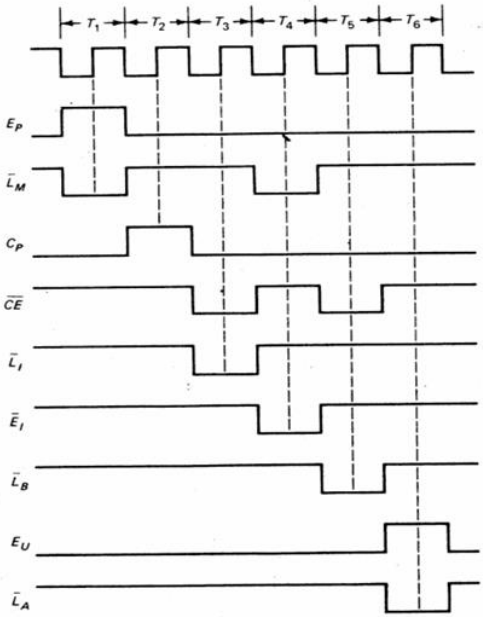
Selama keadaan T6, Eu dan L A yang menjadi aktif. Dengan ini, bagian penjumlah-pengurang (Add/Sub) mempersiapkan akumulator (lihat gambar

6c). Pada pertengah keadaan ini, tepi positif detak mengisikan hasil jumlahan kedalam akumulator.

Kebetulan waktu siap (set up time) dan waktu tunda propagasi dalam rangkaian mempunyai harga yang tepat mencegah terjadinya keadaan pacu pada akumulator selama keadaan eksekusi terakhir. Bilamana tepi positif sinyal detak muncul dalam keadaan gambar 6c, maka isi akumulator akan berubah dan menyebabkan isi penjumlah pengurang mengalami perubahan pula. Isi yang baru ini akan beredar kembali kemasukan akumulator, akan tetapi isi itu tidak akan tiba disana sebelum berlalunya dua kali waktu tunda propagasi sesudah munculnya tepi positif dari pulsa detak (satu waktu tunda propagasi pada akumulator dan satu lagi terjadi pada penjumlah-pengurang). Saat tibanya isi baru itu sudah terlalu terlembat untuk mempersiapkan akumulator. Akibatnya, akumulator terhindar dari keadaan pacu (yaitu pengisian lebih dari sekali pada tepi pulsa detak yang sama).

Gambar 7 memperlihatkan diagram pewaktuan untuk siklus pengambilan dan rutin ADD. Rutin pengambilan ini serupa dengan yang telah dibahas sebelumnya; keadaan T1 mengisikan alamat PC kedalam MAR; keadaan T2 menaikkan hitungan

pencacah program; keadaan T3 mengirimkan instruksi dari lokasi menuju ke register instruksi.



Gambar 7 Diagram Pewaktuan untuk Rutin ADD

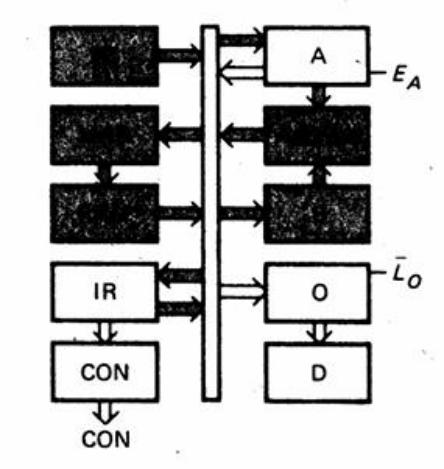
Selama keadaan T4, E 1dan L M menjadi aktif. Pada tepi pulsa detak berikutnya, medan alamat didalam register instruksi berpindah ke MAR. Selama keadaan T5 CE, dan L B yang aktif. Oleh karena itu, kata RAM yang dialamatkan itu, diisikan kedalam register B dipertengahan keadaan ini. Selama keadaan T6 E Udan L Amenjadi aktif, dan pada waktu tibanya tepi poositif, hasil jumlahan dari penjumlah-pengurang didalam akumulator.

C. Rutin SUB

Rutin Sub serupa dengan rutin ADD. Gambar 6a dan b melukiskan bagian-bagian yang aktif dari SAP-1 selama keadaan T4 dan T5. Selamna keadaan T6 , SU yang tinggi disalurkan kepada bagian penjumlah-pengurang dari gambar 6c. Diagram pewaktuan yang bersangkutan hampir identik dengan gambar 7. Bayangkan bahwa SU rendah selama keadaan T1 sampai T5 dan SU menjadi tinggi selama keadaan T6.

D. Rutin OUT

Andaikan bahwa register instruksi berisi instruksi OUT pada akhir sebuah siklus pengambilan. Maka medan instruksi akan memasuki pengendali-pengurut guna pendekodeannya. Kemudian pengendali-pengurut mengeluarkan kata-kendali yang diperlukan mengeluarkan isi akumulator ke register keluaran.



Gambar 8 Keadaan T_4 instruksi OUT

Gambar 8 memperlihatkan bagian-bagian aktif dari SAP=1 selama eksekusi dari sebuah instruksi out. Sehubungan dengan EA dan L Oyang aktif, tepi positif berikutnya dari sinyal detak akan memindahkan isi akumlator ke register keluaran selama keluaran T4. Pada keadaan T5 dan T6 tidak terjadi operasi (nop).

Dalam gambar 9 ditunjukkan diagram pewaktuan untuk rutin pengambilan dan rutin OUT. Sekali lagi, siklus pengambilannya sama : keadaan alamat, keadaan penambahan dan keadaan memori. Selama keadaan T4, EA dan L O aktif ; dan kata akumulator dipindahkan ke register keluaran pada waktu tibanya tepi positif dari pulsa detak.

E. RUTIN HALT

Instruksi HLT tidak membutuhkan rutin kendali

karena tidak sebuahpun register terlihat dalam eksekusi dalam eksekusi instruksi bersangkutan. Bilamana IR berisi :

IR = 1111 XXXX

Medan instruksi 1111 akan memberi tahu pengendali-pengurut untuk menghentikan pemrosesan data. Pengendali-pengurut memberhentikan operasi komputer dengan menghentikan detak.

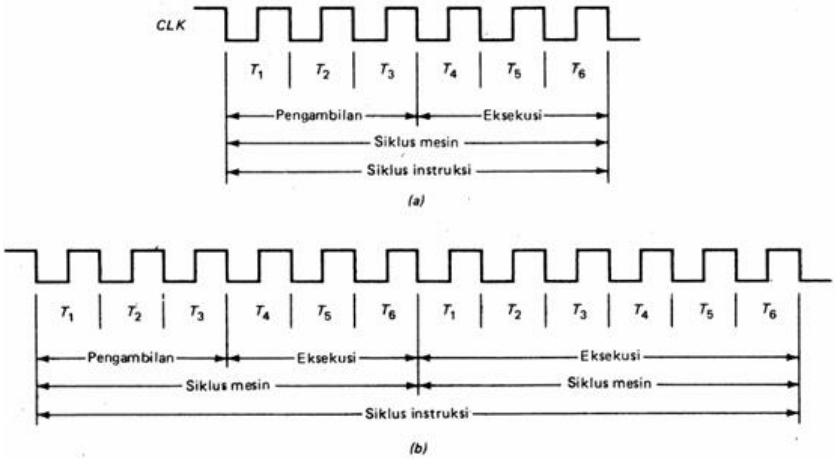
TABEL 1. PERANGKAT INSTRUKSI SAP-1

Mnemonik	Operasi
LDA	Isikan data RAM ke dalam akumulator
ADD	Tambahkan data RAM pada akumulator
SUB	Kurangkan data RAM dari akumulator
OUT	Isikan data akumulator ke dalam register keluaran
HLT	Hentikan pemrosesan

F. Siklus Mesin dan Siklus Instruksi

SAP-1 memiliki enam keadaan T (tiga untuk pengambilan dan tiga untuk eksekusi). Enam keadaan tersebut dinamakan satu siklus mesin (lihat gambar 10a). Komputer membutuhkan satu siklus mesin untuk mengambil dan melaksanakan setiap instruksi. Pendetak SAP-1 berprekuensi KHz yang ekivalen

dengan periode 1 ms. Dengan demikian, satu siklus mesin SAP-1 memerlukan waktu 6 ms. Mesin SAP-1 memerlukan waktu 6 ms.



Gambar 10. (a) Siklus Instruksi SAP-1 (b) Siklus Instruksi untuk Dua siklus Mesin

SAP-2 agak berbeda dari SAP-1 karena beberapa dari instruksinya memerlukan lebih dari satu siklus mesin untuk pengambilan dan eksekusinya gambar 10b memperlihatkan diagram pewaktuan bagi instruksi yang memerlukan dua siklus mesin. Tiga keadaan T yang pertama adalah siklus pengambilan, namun untuk eksekusinya diperlukan sembilan keadaan T berikutnya. Ini disebabkan oleh karena instruksi dua siklus mesin lebih rumit sehingga memerlukan keadaan T ekstra untuk menyelesaikan eksekusinya.

Jumlah keadaan T yang diperlukan guna mengambil dan melaksanakan sebuah instruksi dinamakan siklus

instruksi (instruction cycle) . Dalam SAP-1, siklus instruksi sama dengan siklus mesin. Dalam SAP-2 dan mikrokomputer yang lain, siklus instruksi bisa sama dengan dua siklus mesin atau lebih, seperti ditunjukkan pada gambar 10b.

9.3. Tugas Pendahuluan:

a. Tuliskan tujuan praktikum

   Tujuan praktikum ini adalah agar mahasiswa memahami dan dapat membuat program berdasarkan siklus pelaksanaan instruksi serta instruksi dasar SAP-1 seperti LDA, ADD, SUB, OUT, dan HLT.

b. Jelaskan tentang Siklus Pelaksanaan

   Siklus pelaksanaan adalah rangkaian langkah setelah instruksi diambil, terdiri dari T4 hingga T6. Pada T4, alamat diproses; T5 digunakan untuk transfer data; dan T6 biasanya merupakan kondisi NOP atau tidak ada operasi.

c. Apa yang anda ketahui tentang fungsi utama Set Instruksi SAP-1

   Instruksi LDA memindahkan data dari memori ke akumulator. ADD menjumlahkan data dari memori ke akumulator. SUB mengurangi isi memori dari akumulator. OUT mengirim data dari akumulator ke output. HLT menghentikan semua operasi komputer.

9.4. Percobaan:

1. Membuat program Siklus Pelaksanaan LDA

Program:

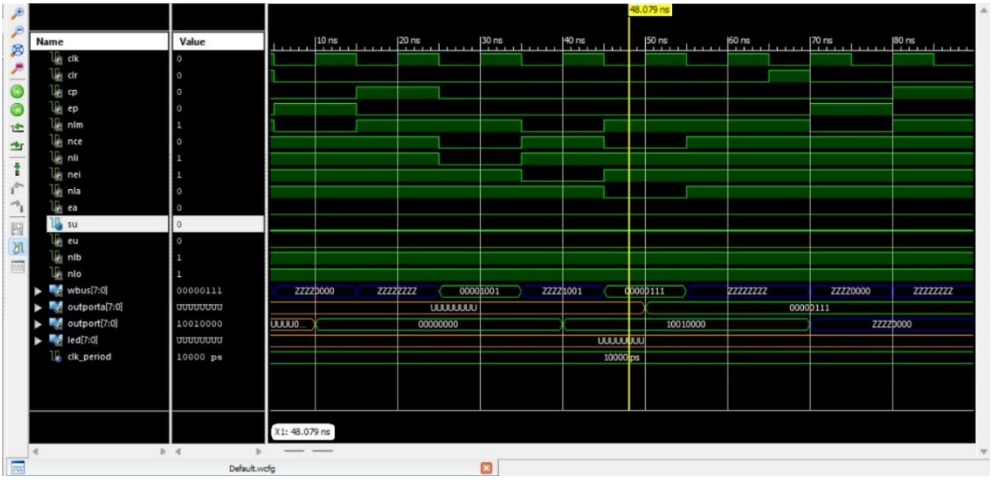
```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY sap1_tb IS
END sap1_tb;

ARCHITECTURE behavior OF sap1_tb IS

    COMPONENT SAP1
        PORT (
            CLK       : IN  std_logic;
            CLR       : IN  std_logic;
            Cp        : IN  std_logic;
            Ep        : IN  std_logic;
            nLm       : IN  std_logic;
            nCe       : IN  std_logic;
            nLi       : IN  std_logic;
            nEi       : IN  std_logic;
            nLa       : IN  std_logic;
            Ea        : IN  std_logic;
            Su        : IN  std_logic;
            Eu        : IN  std_logic;
            nLb       : IN  std_logic;
            nLo       : IN  std_logic;
            wbus      : OUT std_logic_vector(7 downto 0);
            outPortA  : OUT std_logic_vector(7 downto 0);
            outPort   : OUT std_logic_vector(7 downto 0);
            LED       : OUT std_logic_vector(7 downto 0)
        );
    END COMPONENT;

    -- Inputs
    signal CLK      : std_logic := '0';
    signal CLR      : std_logic := '0';
    signal Cp       : std_logic := '0';
    signal Ep       : std_logic := '0';
    signal nLm      : std_logic := '1';
    signal nCe      : std_logic := '1';
    signal nLi      : std_logic := '1';
    signal nEi      : std_logic := '1';
    signal nLa      : std_logic := '1';
    signal Ea       : std_logic := '0';
    signal Su       : std_logic := '0';
    signal Eu       : std_logic := '0';
```

```
    signal nLb      : std_logic := '1';
    signal nLo      : std_logic := '1';

    -- Outputs
    signal wbus       : std_logic_vector(7 downto 0) :=
"00000000";
    signal outPortA  : std_logic_vector(7 downto 0) :=
"00000000";
    signal outPort   : std_logic_vector(7 downto 0) :=
"00000000";
    signal LED        : std_logic_vector(7 downto 0) :=
"00000000";

    -- Clock period definition
    constant CLK_period : time := 10 ns;

BEGIN

    -- Instantiate the Unit Under Test (UUT)
    uut: SAP1 PORT MAP (
        CLK       => CLK,
        CLR       => CLR,
        Cp        => Cp,
        Ep        => Ep,
        nLm       => nLm,
        nCe       => nCe,
        nLi       => nLi,
        nEi       => nEi,
        nLa       => nLa,
        Ea        => Ea,
        Su        => Su,
        Eu        => Eu,
        nLb       => nLb,
        nLo       => nLo,
        wbus      => wbus,
        outPortA  => outPortA,
        outPort   => outPort,
        LED       => LED
    );

    -- Clock process definition
    CLK_process : process
    begin
        CLK <= '1';
        wait for CLK_period / 2;
        CLK <= '0';
        wait for CLK_period / 2;
    end process;
```

```
-- Stimulus process
stim_proc: process
begin
    CLR <= '1';
    wait for CLK_period / 2;
    CLR <= '0';
    -- *****  LDA *****
    -- T1
    Cp  <= '0';  Ep  <= '1';
    nLm <= '0';
    nCe <= '1';
    nLi <= '1';  nEi <= '1';
    nLa <= '1';  Ea  <= '0';
    Su  <= '0';  Eu  <= '0';
    nLb <= '1';
    nLo <= '1';
    wait for CLK_period;
    -- T2
    Cp  <= '1';  Ep  <= '0';
    nLm <= '1';
    nCe <= '1';
    nLi <= '1';  nEi <= '1';
    nLa <= '1';  Ea  <= '0';
    Su  <= '0';  Eu  <= '0';
    nLb <= '1';
    nLo <= '1';
    wait for CLK_period;
    -- T3
    Cp  <= '0';  Ep  <= '0';
    nLm <= '1';
    nCe <= '0';
    nLi <= '0';  nEi <= '1';
    nLa <= '1';  Ea  <= '0';
    Su  <= '0';  Eu  <= '0';
    nLb <= '1';
    nLo <= '1';
    wait for CLK_period;
    -- T4
    Cp  <= '0';  Ep  <= '0';
    nLm <= '0';
    nCe <= '1';
    nLi <= '1';  nEi <= '0';
    nLa <= '1';  Ea  <= '0';
    Su  <= '0';  Eu  <= '0';
    nLb <= '1';
    nLo <= '1';
    wait for CLK_period;
```

```
            -- T5
            Cp  <= '0';  Ep  <= '0';
            nLm <= '1';
            nCe <= '0';
            nLi <= '1';  nEi <= '1';
            nLa <= '0';  Ea  <= '0';
            Su  <= '0';  Eu  <= '0';
            nLb <= '1';
            nLo <= '1';
            wait for CLK_period;
            -- T6
            Cp  <= '0';  Ep  <= '0';
            nLm <= '1';
            nCe <= '1';
            nLi <= '1';  nEi <= '1';
            nLa <= '1';  Ea  <= '0';
            Su  <= '0';  Eu  <= '0';
            nLb <= '1';
            nLo <= '1';
            wait for CLK_period;
        end process;
END behavior;
```

Hasil:



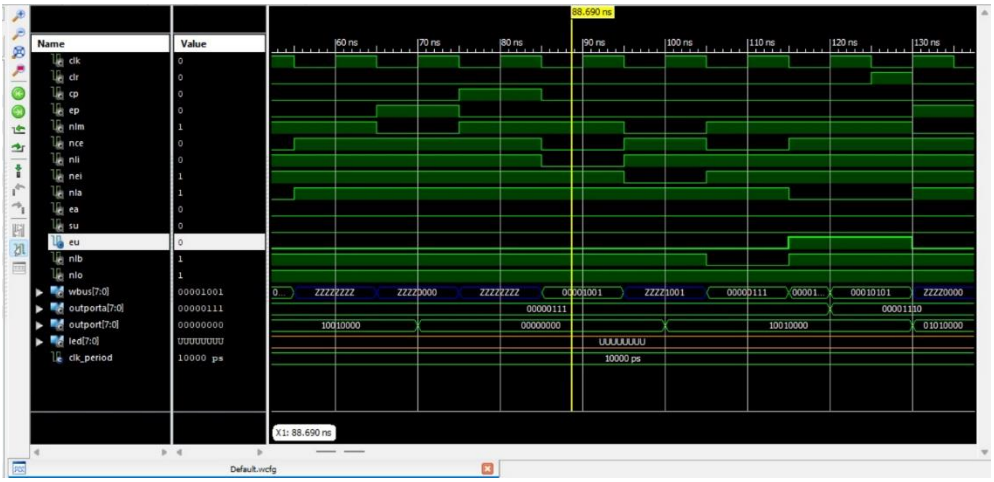
2. Membuat program Siklus Pelaksanaan ADD

Program:

```
-- *****  ADD ***************
  -- T1
  Cp    <= '0';  Ep    <= '1';
  nLm   <= '0';
  nCe   <= '1';
  nLi   <= '1';  nEi   <= '1';
  nLa   <= '1';  Ea    <= '0';
  Su    <= '0';  Eu    <= '0';
  nLb   <= '1';
  nLo   <= '1';
  wait for CLK_period;

  -- T2
  Cp    <= '1';  Ep    <= '0';
  nLm   <= '1';
  nCe   <= '1';
  nLi   <= '1';  nEi   <= '1';
  nLa   <= '1';  Ea    <= '0';
  Su    <= '0';  Eu    <= '0';
  nLb   <= '1';
  nLo   <= '1';
  wait for CLK_period;

  -- T3
  Cp    <= '0';  Ep    <= '0';
  nLm   <= '1';
  nCe   <= '0';
  nLi   <= '0';  nEi   <= '1';
  nLa   <= '1';  Ea    <= '0';
  Su    <= '0';  Eu    <= '0';
  nLb   <= '1';
  nLo   <= '1';
  wait for CLK_period;

  -- T4
  Cp    <= '0';  Ep    <= '0';
  nLm   <= '0';
  nCe   <= '1';
  nLi   <= '1';  nEi   <= '0';
  nLa   <= '1';  Ea    <= '0';
  Su    <= '0';  Eu    <= '0';
  nLb   <= '1';
  nLo   <= '1';
  wait for CLK_period;
```

```
-- T5
Cp    <= '0';  Ep    <= '0';
nLm   <= '1';
nCe   <= '0';
nLi   <= '1';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '0';
nLo   <= '1';
wait for CLK_period;

-- T6
Cp    <= '0';  Ep    <= '0';
nLm   <= '1';
nCe   <= '1';
nLi   <= '1';  nEi   <= '1';
nLa   <= '0';  Ea    <= '0';
Su    <= '0';  Eu    <= '1';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;
```

Hasil:



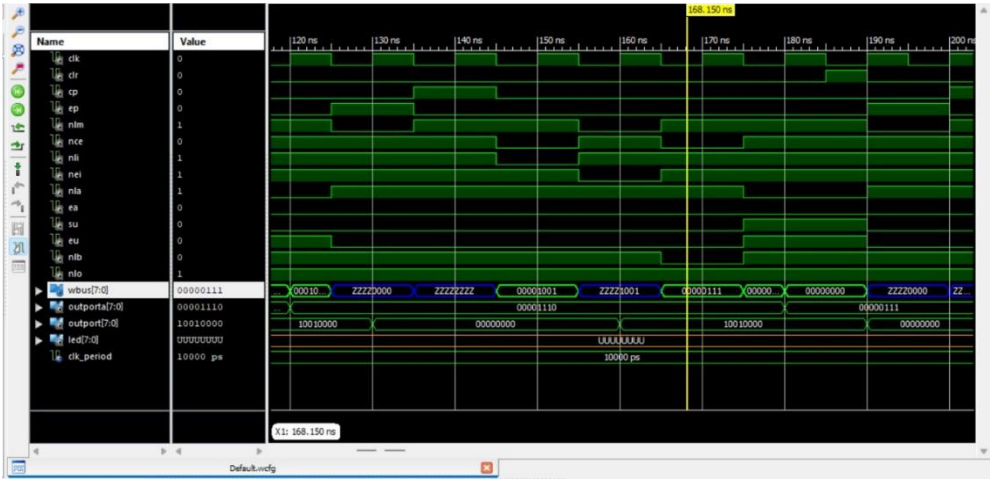
3. Membuat program Siklus Pelaksanaan SUB

Program:

```
-- *****  SUBB ******************
-- T1
Cp    <= '0';  Ep    <= '1';
nLm   <= '0';
nCe   <= '1';
nLi   <= '1';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;

-- T2
Cp    <= '1';  Ep    <= '0';
nLm   <= '1';
nCe   <= '1';
nLi   <= '1';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;

-- T3
Cp    <= '0';  Ep    <= '0';
nLm   <= '1';
nCe   <= '0';
nLi   <= '0';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;

-- T4
Cp    <= '0';  Ep    <= '0';
nLm   <= '0';
nCe   <= '1';
nLi   <= '1';  nEi   <= '0';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;
```

```
-- T5
Cp     <= '0';  Ep     <= '0';
nLm    <= '1';
nCe    <= '0';
nLi    <= '1';  nEi    <= '1';
nLa    <= '1';  Ea     <= '0';
Su     <= '0';  Eu     <= '0';
nLb    <= '0';
nLo    <= '1';
wait for CLK_period;

-- T6
Cp     <= '0';  Ep     <= '0';
nLm    <= '1';
nCe    <= '1';
nLi    <= '1';  nEi    <= '1';
nLa    <= '0';  Ea     <= '0';
Su     <= '1';  Eu     <= '1';
nLb    <= '1';
nLo    <= '1';
wait for CLK_period;
```

Hasil:



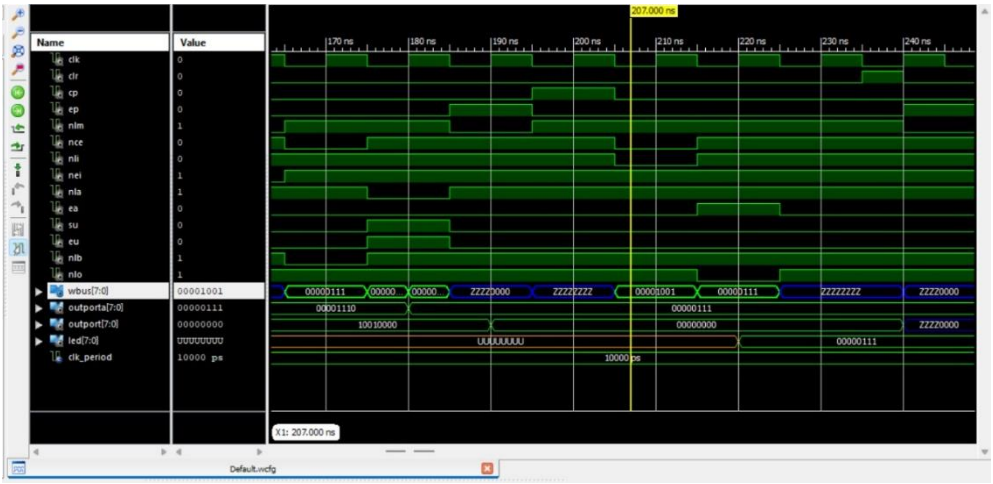
4. Membuat program Siklus Pelaksanaan OUT

Program:

```
-- ***** OUT ***************
-- T1
Cp    <= '0';  Ep    <= '1';
nLm   <= '0';
nCe   <= '1';
nLi   <= '1';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;

-- T2
Cp    <= '1';  Ep    <= '0';
nLm   <= '1';
nCe   <= '1';
nLi   <= '1';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;

-- T3
Cp    <= '0';  Ep    <= '0';
nLm   <= '1';
nCe   <= '0';
nLi   <= '0';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;

-- T4
Cp    <= '0';  Ep    <= '0';
nLm   <= '1';
nCe   <= '1';
nLi   <= '1';  nEi   <= '1';
nLa   <= '1';  Ea    <= '1';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '0';
wait for CLK_period;
```

```
-- OFF
Cp     <= '0';  Ep     <= '0';
nLm    <= '1';
nCe    <= '1';
nLi    <= '1';  nEi    <= '1';
nLa    <= '1';  Ea     <= '0';
Su     <= '0';  Eu     <= '0';
nLb    <= '1';
nLo    <= '1';
wait for CLK_period;
```

Hasil:



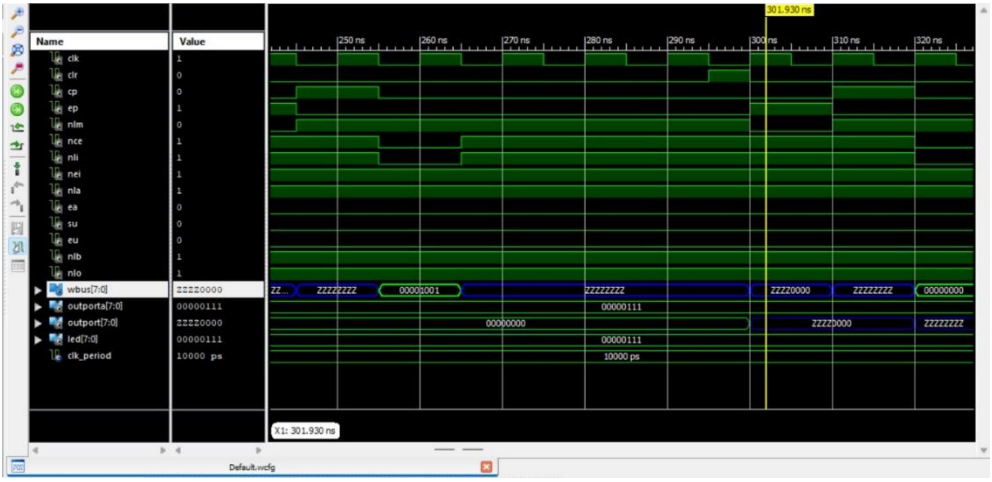
5. Membuat program Siklus Pelaksanaan HLT

Program:

```
-- ***** HALT ****************
-- T1
Cp    <= '0';  Ep    <= '1';
nLm   <= '0';
nCe   <= '1';
nLi   <= '1';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;

-- T2
Cp    <= '1';  Ep    <= '0';
nLm   <= '1';
nCe   <= '1';
nLi   <= '1';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;

-- T3
Cp    <= '0';  Ep    <= '0';
nLm   <= '1';
nCe   <= '0';
nLi   <= '0';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
nLo   <= '1';
wait for CLK_period;

-- T4
Cp    <= '0';  Ep    <= '0';
nLm   <= '1';
nCe   <= '1';
nLi   <= '1';  nEi   <= '1';
nLa   <= '1';  Ea    <= '0';
Su    <= '0';  Eu    <= '0';
nLb   <= '1';
```

```
nLo    <= '1';
wait for CLK_period;

-- T5
Cp     <= '0';  Ep     <= '0';
nLm    <= '1';
nCe    <= '1';
nLi    <= '1';  nEi    <= '1';
nLa    <= '1';  Ea     <= '0';
Su     <= '0';  Eu     <= '0';
nLb    <= '1';
nLo    <= '1';
wait for CLK_period;

-- T6
Cp     <= '0';  Ep     <= '0';
nLm    <= '1';
nCe    <= '1';
nLi    <= '1';  nEi    <= '1';
nLa    <= '1';  Ea     <= '0';
Su     <= '0';  Eu     <= '0';
nLb    <= '1';
nLo    <= '1';
wait for CLK_period;
```

Hasil:



9.5.Latihan

a. Buatlah program sesuai dengan soal berikut : 4 + 5 + 6 – 7 => OUT, HLT

Program:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY sap1_tb_latihan IS
END ENTITY sap1_tb_latihan;
ARCHITECTURE behavior OF sap1_tb_latihan IS
    -- Component Declaration for the Unit Under Test (UUT)
    COMPONENT SAP1
        PORT (
            CLK      : IN  STD_LOGIC;
            CLR      : IN  STD_LOGIC;
            Cp       : IN  STD_LOGIC;
            Ep       : IN  STD_LOGIC;
            nLm      : IN  STD_LOGIC;
            nCe      : IN  STD_LOGIC;
            nLi      : IN  STD_LOGIC;
            nEi      : IN  STD_LOGIC;
            nLa      : IN  STD_LOGIC;
            Ea       : IN  STD_LOGIC;
            Su       : IN  STD_LOGIC;
            Eu       : IN  STD_LOGIC;
            nLb      : IN  STD_LOGIC;
            nLo      : IN  STD_LOGIC;
            wbus     : OUT STD_LOGIC_VECTOR(7 downto 0);
            outPortA : OUT STD_LOGIC_VECTOR(7 downto 0);
            outPort  : OUT STD_LOGIC_VECTOR(7 downto 0);
            LED      : OUT STD_LOGIC_VECTOR(7 downto 0)
        );
    END COMPONENT SAP1;
    -- Inputs
    signal CLK      : STD_LOGIC := '0';
    signal CLR      : STD_LOGIC := '0';
    signal Cp       : STD_LOGIC := '0';
    signal Ep       : STD_LOGIC := '0';
    signal nLm      : STD_LOGIC := '1';
    signal nCe      : STD_LOGIC := '1';
    signal nLi      : STD_LOGIC := '1';
    signal nEi      : STD_LOGIC := '1';
```

```
    signal nLa      : STD_LOGIC := '1';
    signal Ea       : STD_LOGIC := '0';
    signal Su       : STD_LOGIC := '0';
    signal Eu       : STD_LOGIC := '0';
    signal nLb      : STD_LOGIC := '1';
    signal nLo      : STD_LOGIC := '1';
    -- Outputs
    signal wbus       : STD_LOGIC_VECTOR(7 downto 0) :=
"00000000";
    signal outPortA  :  STD_LOGIC_VECTOR(7 downto 0) :=
"00000000";
    signal outPort   :  STD_LOGIC_VECTOR(7 downto 0) :=
"00000000";
    signal LED        : STD_LOGIC_VECTOR(7 downto 0) :=
"00000000";


    -- Clock period definitions
    constant CLK_period : TIME := 10 ns;


BEGIN
    -- Instantiate the Unit Under Test (UUT)
    uut : SAP1
        PORT MAP(
            CLK      => CLK,
            CLR      => CLR,
            Cp       => Cp,
            Ep       => Ep,
            nLm      => nLm,
            nCe      => nCe,
            nLi      => nLi,
            nEi      => nEi,
            nLa      => nLa,
            Ea       => Ea,
            Su       => Su,
            Eu       => Eu,
            nLb      => nLb,
            nLo      => nLo,
            wbus     => wbus,
            outPortA => outPortA,
            outPort  => outPort,
            LED      => LED
        );


    -- Clock process definitions
    CLK_process : PROCESS
```

```
    BEGIN
        CLK <= '1';
        WAIT FOR CLK_period / 2;
        CLK <= '0';
        WAIT FOR CLK_period / 2;
    END PROCESS CLK_process;


    -- Stimulus process
    stim_proc : PROCESS
    BEGIN
        -- Initialize Inputs
        CLR <= '1';
        WAIT FOR CLK_period / 2;
        CLR <= '0';
        WAIT FOR CLK_period; -- Allow CLR to settle


        --  *******  LDA  4  (Load  4  into  Accumulator)
**********
        -- T1: PC -> BUS, MAR_in <- BUS (address 0)
        report "LDA 4: T1 - Fetch address";
        Cp    <= '0'; Ep <= '1'; nLm <= '0'; nCe <= '1';
nLi <= '1';
        WAIT FOR CLK_period;


        -- T2: PC increment (now points to address 1)
        report "LDA 4: T2 - PC increment";
        Cp    <= '1'; Ep <= '0'; nLm <= '1';
        WAIT FOR CLK_period;


        -- T3: RAM -> BUS (LDA instruction), IR_in <- BUS
        report "LDA 4: T3 - Fetch instruction";
        Cp    <= '0'; nCe <= '0'; nLi <= '0';
        WAIT FOR CLK_period;


        -- T4: IR_low_nibble -> BUS (operand address 4),
MAR_in <- BUS
        report "LDA 4: T4 - Fetch operand address";
        nLm <= '0'; nCe <= '1'; nLi <= '1'; nEi <= '0';
        WAIT FOR CLK_period;


        -- T5: RAM -> BUS (value 4), RegA_in <- BUS
        report "LDA 4: T5 - Load value into Accumulator";
```

```
        nLm <= '1'; nCe <= '0'; nLa <= '0';
        WAIT FOR CLK_period;


        -- T6: Idle cycle
        report "LDA 4: T6 - Idle";
        nCe <= '1'; nLa <= '1';
        WAIT FOR CLK_period;


        --  ********  ADD  5  (Add  5  to  Accumulator)
************
        -- T1: PC -> BUS (address 1), MAR_in <- BUS
        report "ADD 5: T1 - Fetch address";
        Cp <= '0'; Ep <= '1'; nLm <= '0';
        WAIT FOR CLK_period;


        -- T2: PC increment (now points to address 2)
        report "ADD 5: T2 - PC increment";
        Cp <= '1'; Ep <= '0'; nLm <= '1';
        WAIT FOR CLK_period;


        -- T3: RAM -> BUS (ADD instruction), IR_in <- BUS
        report "ADD 5: T3 - Fetch instruction";
        Cp <= '0'; nCe <= '0'; nLi <= '0';
        WAIT FOR CLK_period;


        -- T4: IR_low_nibble -> BUS (operand address 5),
MAR_in <- BUS
        report "ADD 5: T4 - Fetch operand address";
        nLm <= '0'; nCe <= '1'; nLi <= '1'; nEi <= '0';
        WAIT FOR CLK_period;


        -- T5: RAM -> BUS (value 5), RegB_in <- BUS
        report "ADD 5: T5 - Load value into RegB";
        nLm <= '1'; nCe <= '0'; nLb <= '0';
        WAIT FOR CLK_period;


        --  T6:  ALU  operation  (A  +  B),  result  ->
Accumulator
        report "ADD 5: T6 - Perform addition";
        nLb <= '1'; nLa <= '0'; Eu <= '1'; Su <= '0';
        WAIT FOR CLK_period;
```

```
        --  ***********  ADD  6  (Add  6  to  Accumulator)
**********
        -- T1: PC -> BUS (address 2), MAR_in <- BUS
        report "ADD 6: T1 - Fetch address";
        nLa <= '1'; Eu <= '0';
        Cp <= '0'; Ep <= '1'; nLm <= '0';
        WAIT FOR CLK_period;


        -- T2: PC increment (now points to address 3)
        report "ADD 6: T2 - PC increment";
        Cp <= '1'; Ep <= '0'; nLm <= '1';
        WAIT FOR CLK_period;


        -- T3: RAM -> BUS (ADD instruction), IR_in <- BUS
        report "ADD 6: T3 - Fetch instruction";
        Cp <= '0'; nCe <= '0'; nLi <= '0';
        WAIT FOR CLK_period;


        -- T4: IR_low_nibble -> BUS (operand address 6),
MAR_in <- BUS
        report "ADD 6: T4 - Fetch operand address";
        nLm <= '0'; nCe <= '1'; nLi <= '1'; nEi <= '0';
        WAIT FOR CLK_period;


        -- T5: RAM -> BUS (value 6), RegB_in <- BUS
        report "ADD 6: T5 - Load value into RegB";
        nLm <= '1'; nCe <= '0'; nLb <= '0';
        WAIT FOR CLK_period;


        --  T6:  ALU  operation  (A  +  B),  result  ->
Accumulator
        report "ADD 6: T6 - Perform addition";
        nLb <= '1'; nLa <= '0'; Eu <= '1'; Su <= '0';
        WAIT FOR CLK_period;


        -- ********** SUB 7 (Subtract 7 from Accumulator)
****
        -- T1: PC -> BUS (address 3), MAR_in <- BUS
        report "SUB 7: T1 - Fetch address";
        nLa <= '1'; Eu <= '0';
```

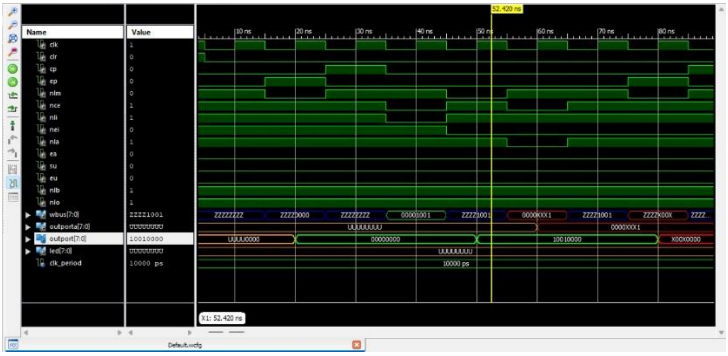
```
        Cp <= '0'; Ep <= '1'; nLm <= '0';
        WAIT FOR CLK_period;


        -- T2: PC increment (now points to address 4)
        report "SUB 7: T2 - PC increment";
        Cp <= '1'; Ep <= '0'; nLm <= '1';
        WAIT FOR CLK_period;


        -- T3: RAM -> BUS (SUB instruction), IR_in <- BUS
        report "SUB 7: T3 - Fetch instruction";
        Cp <= '0'; nCe <= '0'; nLi <= '0';
        WAIT FOR CLK_period;


        -- T4: IR_low_nibble -> BUS (operand address 7),
MAR_in <- BUS
        report "SUB 7: T4 - Fetch operand address";
        nLm <= '0'; nCe <= '1'; nLi <= '1'; nEi <= '0';
        WAIT FOR CLK_period;


        -- T5: RAM -> BUS (value 7), RegB_in <- BUS
        report "SUB 7: T5 - Load value into RegB";
        nLm <= '1'; nCe <= '0'; nLb <= '0';
        WAIT FOR CLK_period;


        --  T6:  ALU  operation  (A  -  B),  result  ->
Accumulator
        report "SUB 7: T6 - Perform subtraction";
        nLb <= '1'; nLa <= '0'; Eu <= '1'; Su <= '1';
        WAIT FOR CLK_period;


        --  *******  OUT  (Output  Accumulator  value)
*************
        -- T1: PC -> BUS (address 4), MAR_in <- BUS
        report "OUT: T1 - Fetch address";
        nLa <= '1'; Eu <= '0';
        Cp <= '0'; Ep <= '1'; nLm <= '0';
        WAIT FOR CLK_period;


        -- T2: PC increment (now points to address 5)
        report "OUT: T2 - PC increment";
        Cp <= '1'; Ep <= '0'; nLm <= '1';
```

```
        WAIT FOR CLK_period;


        -- T3: RAM -> BUS (OUT instruction), IR_in <- BUS
        report "OUT: T3 - Fetch instruction";
        Cp <= '0'; nCe <= '0'; nLi <= '0';
        WAIT FOR CLK_period;


        -- T4: Accumulator -> Output Register
        report "OUT: T4 - Output value";
        nCe <= '1'; nLi <= '1'; Ea <= '1'; nLo <= '0';
        WAIT FOR CLK_period;


        -- Idle cycle
        report "OUT: T5 - Idle";
        Ea <= '0'; nLo <= '1';
        WAIT FOR CLK_period;


        --     **********     HLT     (Halt     execution)
********************
        -- T1: PC -> BUS (address 5), MAR_in <- BUS
        report "HLT: T1 - Fetch address";
        Cp <= '0'; Ep <= '1'; nLm <= '0';
        WAIT FOR CLK_period;


        -- T2: PC increment (now points to address 6)
        report "HLT: T2 - PC increment";
        Cp <= '1'; Ep <= '0'; nLm <= '1';
        WAIT FOR CLK_period;


        -- T3: RAM -> BUS (HLT instruction), IR_in <- BUS
        report "HLT: T3 - Fetch instruction";
        Cp <= '0'; nCe <= '0'; nLi <= '0';
        WAIT FOR CLK_period;


        -- T4-T6: Halt state - no operations
        report "HLT: T4 - Halt";
        nCe <= '1'; nLi <= '1';
        WAIT FOR CLK_period;

        report "HLT: T5 - Halt";
        WAIT FOR CLK_period;
```

```
            report "HLT: T6 - Halt";
            WAIT FOR CLK_period;


            WAIT; -- Will wait forever
        END PROCESS stim_proc;
    END ARCHITECTURE behavior;
```

b. Tampilkan hasil simulasinya.

Hasil:



9.6.Analisa

Pada praktikum ini kita membuat beberapa program, yang pertama yaitu program LDA. Program di atas adalah sebuah testbench VHDL untuk menguji rangkaian SAP-1 (Simple As Possible Computer 1). Testbench ini bertujuan untuk mensimulasikan perilaku rangkaian SAP1 dengan memberikan input sinyal-sinyal kontrol secara berurutan, sesuai dengan siklus eksekusi instruksi **LDA (Load Accumulator)**. Dalam blok arsitektur behavior, sebuah komponen SAP1 dideklarasikan dengan berbagai sinyal

input dan output, lalu diinstansiasi dan dihubungkan dengan sinyal-sinyal internal testbench. Testbench ini juga memiliki proses CLK_process yang menghasilkan sinyal clock (CLK) dengan periode 10 ns, yang berguna untuk menyinkronkan semua proses internal SAP1. Kemudian, proses stim_proc digunakan untuk memberikan rangkaian sinyal kontrol secara bertahap selama enam siklus clock (T1 hingga T6) yang merepresentasikan siklus fetch dan eksekusi instruksi **LDA**. Proses ini dimulai dengan mengaktifkan sinyal CLR untuk mereset sistem, kemudian memberikan kombinasi sinyal seperti Cp, Ep, nLm, nCe, nLi, nEi, nLa, dan Ea pada setiap siklus. Perubahan sinyal ini bertujuan untuk mengontrol jalannya data antar register, memori, dan ALU di dalam SAP1. Misalnya, pada T1 dan T2 program counter (PC) digunakan untuk mengambil alamat instruksi, kemudian pada T3 dan T4 instruksi dibaca dari memori dan dimuat ke instruction register, dan pada T5 dan T6 operand dari memori dimuat ke akumulator (A). Secara keseluruhan, testbench ini berfungsi untuk memastikan bahwa instruksi **LDA** dijalankan dengan benar oleh SAP1, dan sinyal-sinyal kontrol yang diberikan mencerminkan mikroinstruksi SAP1 pada tiap tahap siklus mesin.

Yang kedua adalah program ADD Blok kode VHDL di atas merupakan bagian dari *stimulus process* yang mensimulasikan eksekusi instruksi **ADD** dalam sistem SAP-1. Instruksi **ADD** bertujuan untuk menambahkan data dari alamat memori yang ditunjuk oleh operand ke register akumulator (A). Proses dimulai dari siklus T1 hingga T6, yang masing-masing merepresentasikan satu langkah dalam mikrosiklus eksekusi instruksi.

Pada **T1**, program counter (PC) diaktifkan dengan sinyal Ep = '1' untuk menempatkan alamat instruksi ke bus, lalu sinyal nLm = '0' mengizinkan data tersebut disimpan ke memory address register (MAR). Pada **T2**, PC dinaikkan (increment) dengan mengaktifkan Cp = '1'. Kemudian pada **T3**, sinyal nCe = '0' dan nLi = '0' memungkinkan instruksi dari memori dimuat ke instruction register (IR). Pada **T4**, IR mengeluarkan operand (alamat data) ke bus dengan nEi = '0', dan operand itu dimuat ke MAR dengan nLm = '0'.

Di **T5**, data dari memori yang ditunjuk oleh MAR dikeluarkan ke bus dan dimuat ke register B dengan nLb = '0'. Terakhir, pada **T6**, proses penjumlahan terjadi di dalam ALU. Nilai dari akumulator dan register B dijumlahkan, dengan Su = '0' (operasi ADD), dan hasilnya dikembalikan ke akumulator (A) melalui bus dengan Eu

= '1' dan nLa = '0'.

Seluruh rangkaian kontrol ini menggambarkan langkah-langkah mikroprogram dari instruksi ADD pada SAP-1 secara sistematis, dan memungkinkan verifikasi perilaku rangkaian digital SAP1 melalui simulasi.

Yang ketiga adalah program SUB, Blok kode di atas merupakan bagian dari simulasi testbench VHDL untuk instruksi **SUBB (Subtract B from A)** pada sistem SAP-1. Instruksi ini bertujuan untuk mengambil data dari alamat memori tertentu (yang ditunjuk oleh operand), memuatnya ke register B, lalu melakukan pengurangan nilai register B dari akumulator (A), dan menyimpan hasilnya kembali ke A. Eksekusi dilakukan melalui enam siklus clock berturut-turut (T1–T6), yang merepresentasikan urutan mikrooperasi dalam arsitektur SAP-1. Pada **T1**, program counter (PC) mengeluarkan alamat instruksi ke bus (Ep = '1'), dan alamat ini dimuat ke memory address register (MAR) dengan nLm = '0'. Selama itu, sinyal kontrol lainnya diset ke kondisi default agar tidak mengganggu jalannya data. **T2** digunakan untuk menaikkan nilai PC (Cp = '1'). Kemudian pada **T3**, sinyal nCe = '0' dan nLi = '0' menyebabkan data instruksi dari memori dimuat ke instruction register (IR).

Selanjutnya, pada **T4**, operand (alamat data) dikeluarkan dari IR ke bus (nEi = '0') dan dimuat kembali ke MAR (nLm = '0') sebagai penunjuk data yang akan diambil. **T5** digunakan untuk membaca data dari alamat yang telah ditentukan di memori dan memuatnya ke register B (nLb = '0'). Akhirnya, pada **T6**, terjadi proses pengurangan. Data pada register A dan B diproses melalui ALU, dan karena sinyal Su = '1', ALU melakukan operasi **subtraction**. Hasil pengurangan tersebut dikeluarkan (Eu = '1') dan dimuat ke akumulator (A) (nLa = '0'). Dengan konfigurasi sinyal-sinyal ini, testbench mensimulasikan bagaimana instruksi SUBB dijalankan secara rinci, mencerminkan siklus kerja mikroprogram SAP-1, mulai dari pengambilan instruksi, pembacaan operand dari memori, hingga eksekusi pengurangan di ALU.

Pada siklus OUT Blok kode di atas merupakan simulasi dari eksekusi instruksi **OUT** pada sistem SAP-1. Instruksi **OUT** berfungsi untuk menampilkan isi dari **akumulator (A)** ke **port output**. Seperti instruksi lainnya dalam SAP-1, eksekusi instruksi OUT dilakukan melalui beberapa tahap mikrosiklus, dimulai dari T1 hingga T4, diikuti dengan kondisi OFF untuk mengembalikan kontrol ke keadaan idle.

Pada **T1**, program counter (PC) mengeluarkan alamat

instruksi ke bus (Ep = '1'), dan alamat ini disimpan ke **MAR** dengan nLm = '0'. **T2** menaikkan PC (Cp = '1') untuk persiapan instruksi selanjutnya. Pada **T3**, instruksi dibaca dari memori dengan nCe = '0' dan dimuat ke **Instruction Register (IR)** dengan nLi = '0'. Hingga titik ini, urutan sama seperti instruksi lainnya karena merupakan proses **fetch**.

Yang membedakan instruksi OUT terjadi pada **T4**, di mana akumulator (A) dikeluarkan ke bus dengan Ea = '1', dan output port diaktifkan dengan nLo = '0'. Dengan konfigurasi ini, data dari register A dipindahkan ke **output port**, sehingga dapat ditampilkan ke perangkat luar seperti LED atau terminal tampilan. Akhirnya, pada bagian **OFF**, semua sinyal dikembalikan ke keadaan pasif untuk menghindari konflik data pada bus dan menunggu instruksi berikutnya. Secara keseluruhan, instruksi **OUT** berjalan dengan menampilkan isi akumulator ke port output melalui kontrol mikroinstruksi yang tepat, menjadikan data dalam A terlihat di dunia luar sistem SAP-1.

Pada siklus HLT Blok kode di atas merupakan simulasi instruksi **HALT** pada arsitektur SAP-1. Instruksi ini berfungsi untuk **menghentikan eksekusi program**, artinya ketika CPU mencapai instruksi HALT, tidak ada

proses atau instruksi lebih lanjut yang dijalankan.

Seperti biasa, eksekusi dimulai dari **T1 hingga T3** dengan proses **fetch** instruksi: pada T1, alamat dari program counter (PC) dikeluarkan ke bus (Ep = '1') dan dimuat ke memory address register (MAR) (nLm = '0'). Di T2, PC dinaikkan (Cp = '1'). Kemudian pada T3, instruksi dibaca dari memori (nCe = '0') dan dimuat ke instruction register (IR) (nLi = '0').

Mulai dari **T4 hingga T6**, tidak ada aktivitas baru yang dilakukan. Semua sinyal kontrol dalam keadaan pasif (nilai ‘1’ atau ‘0’ yang menunjukkan tidak aktif), sehingga **tidak ada transfer data**, **tidak ada register yang aktif**, dan **tidak ada keluaran**. CPU hanya berputar di mikrosiklus yang tidak melakukan apa-apa — ini menandakan bahwa sistem telah **masuk ke kondisi berhenti (halted state)**. Pada sistem nyata, kondisi ini bisa berarti menunggu **reset**, **interupsi eksternal**, atau **dinyalakan ulang**. Dengan demikian, instruksi **HALT** dalam SAP-1 digunakan untuk **menghentikan siklus instruksi CPU**, biasanya diletakkan di akhir program untuk menandakan bahwa eksekusi program telah selesai.

Pada Latihan soal, Program VHDL di atas merupakan sebuah *testbench* yang dirancang untuk mensimulasikan

perilaku prosesor sederhana SAP-1 (Simple As Possible 1). Tujuan dari program ini adalah untuk menguji bagaimana unit SAP-1 merespons sinyal kontrol selama eksekusi serangkaian instruksi, yaitu LDA, ADD, SUB, OUT, dan HLT. Instruksi-instruksi tersebut dijalankan secara berurutan, dimulai dari instruksi LDA 4 yang memuat nilai dari alamat memori 4 ke dalam akumulator. Kemudian, dilakukan penjumlahan dengan instruksi ADD 5 dan ADD 6, di mana nilai dari masing-masing alamat memori dimuat ke register B dan dijumlahkan dengan nilai yang ada di akumulator. Setelah itu, instruksi SUB 7 digunakan untuk mengurangi nilai di akumulator dengan nilai dari register B yang dimuat dari alamat memori 7. Hasil akhirnya kemudian dikeluarkan menggunakan instruksi OUT, di mana nilai akumulator dikirim ke output register, dan akhirnya instruksi HLT menghentikan seluruh eksekusi program.

Setiap instruksi dieksekusi melalui enam tahap mikrosiklus (T1 sampai T6) dengan pengaturan sinyal kontrol secara manual, seperti Cp, Ep, nLm, nCe, nLi, nEi, dan lainnya. Seluruh proses simulasi ini juga disertai dengan perintah report untuk menunjukkan status eksekusi pada setiap siklus waktu, sehingga mempermudah proses debugging dan pemahaman alur

kerja SAP-1. Ketika disimulasikan, sinyal output seperti outPort atau LED akan menunjukkan hasil akhir dari operasi aritmetika tersebut. Dengan demikian, testbench ini memberikan pemahaman mendalam tentang bagaimana prosesor sederhana bekerja mulai dari pengambilan instruksi, eksekusi, hingga pemberian hasil, serta cocok untuk digunakan sebagai sarana pembelajaran arsitektur komputer dasar dan desain sistem digital.

9.7.Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa simulasi dan analisis menggunakan testbench VHDL terhadap sistem SAP-1 (Simple As Possible Computer 1) berhasil menunjukkan cara kerja dasar dari arsitektur komputer sederhana. Setiap instruksi — mulai dari LDA, ADD, SUB, OUT, hingga HLT — memiliki tahapan mikrosiklus tersendiri (T1 sampai T6) yang merepresentasikan proses fetch, decode, dan execute melalui pengaturan sinyal kontrol secara manual.

Instruksi **LDA** menunjukkan proses pemindahan data dari memori ke akumulator. Instruksi **ADD** dan **SUB** menunjukkan bagaimana operasi aritmetika dilakukan di dalam ALU, dengan data diambil dari memori dan dimuat

ke register B terlebih dahulu. Instruksi **OUT** berfungsi untuk menampilkan hasil dari akumulator ke port output, sedangkan **HLT** berfungsi untuk menghentikan seluruh proses eksekusi. Simulasi juga menegaskan pentingnya pengaturan sinyal kontrol seperti Cp, Ep, nLm, nCe, nLi, nEi, nLa, Ea, nLb, Eu, Su, dan nLo dalam pengelolaan alur data antar komponen internal SAP-1. Dengan menyusun testbench yang tepat, kita dapat memahami bagaimana prosesor memproses instruksi secara bertahap, dari pengambilan alamat hingga eksekusi instruksi.

Secara keseluruhan, praktikum ini memberikan pemahaman konkret tentang prinsip kerja prosesor sederhana dan pentingnya kendali mikroinstruksi dalam sistem digital. Hal ini sangat bermanfaat untuk memperkuat dasar-dasar arsitektur komputer dan desain rangkaian digital berbasis VHDL.